

Final Project Team 2D - Dynamic Pricing

Team Members:

- Richard Horvath
- Adam Pasálek
- Lucas Krämer Sepúlveda
- Aijia Liu
- Zhixian Huang

Context and Objective

We worked with the Dynamic Pricing dataset, which seems to be for a Ride-Sharing Service. Dynamic Pricing includes 1000 records detailing: Number of drivers, Number of riders, Location, Historical Cost, etc. This industry relies on balancing rider demand with driver availability; failing to do so results in loss of potential revenue, high waiting times, and frustrated users who choose to go to the competition. That is the reason why we defined the following business point as highly relevant.

- Are there any times when the demands exceed the availability of the drivers and make the company lose bookings? This also means that there are moments when there are too many drivers and too few customers.
- What happens on long car rides? Does profitability decrease with longer car rides, or does it remain consistent?
- Does the vehicle type and the region affect the Cost per Minute?
- Can we predict the cost of a ride based on real-world variables in the dataset?
- Which features impact the cost of a ride the most?

To understand these problems and find a solution, we decided to first prepare the dataset by cleaning and creating new features that allow us to find deeper and more meaningful insights.

Code Structure

Cleaning Module

The Data preparation is encapsulated in the `clean_data(df: pd.DataFrame)` function. To preserve the integrity of the initial data, the function creates and operates on a copy of the input DataFrame. The internal workflow executes the following steps:

- The function first strips surrounding whitespace from column headers and checks that a predefined set of required numeric and categorical columns is present, raising a `ValueError` if any are missing. Categorical columns are then cast to string types, stripped of whitespace, and formatted to Title Case. Numeric columns are coerced into proper numeric data types.

- Duplicate rows are identified and removed from the dataset. The script then enforces logical boundaries by converting invalid entries such as negative rider or driver counts, sub-zero ride durations, and out-of-bounds ratings into NaN values.
- Before handling missing data, the function creates binary indicator columns to capture operational states. These include `Is_New_Customer` (flagged when past rides equal zero), `Rating_Was_Missing` (flagged before null ratings are overwritten), and `No_Drivers_Available` (flagged when the driver count is zero).
- Missing average ratings are imputed with the column median. Following this, the function drops any rows that contain missing values across a designated subset of essential columns to ensure only complete, valid records pass through to the model.
- Finally, the function calculates practical business metrics. It computes a `Supply_and_Demand` ratio by dividing riders by drivers (assigning NaN where drivers are zero) and a `Cost_per_Minute` metric by dividing the historical cost by the expected ride duration.

Visualizations Module

Categorical Distributions:

- The number of rides across different times of day is calculated using value counts. This data is output as a percentage-based pie chart to show overall distribution.
- Vehicle Types: The total volume of "Economy" versus "Premium" rides is similarly aggregated via value counts and rendered as a standard bar chart.

Supply and Demand Analytics:

- Time and Vehicle Cross-Analysis: A two-dimensional pivot table is constructed to calculate the mean `Supply_and_Demand` ratio. This table is indexed by booking time and segregated by `Vehicle_Type`, then visualized using a Seaborn heatmap with an annotated color gradient to highlight peak constraints.
- Chronological Trends: The `Time_of_Booking` column is explicitly converted into an ordered categorical variable (Morning, Afternoon, Evening, Night) to ensure visual chronological accuracy. The data is grouped by this ordered time, and the average supply/demand ratio is plotted as a bar chart.
- Regional Trends: The dataset is grouped by `Location_Category` (Urban, Suburban, Rural) to calculate and plot the average demand-to-supply ratio in a bar chart.

Pricing and Cost Efficiency:

- Duration vs. Cost: The dataset is dynamically subset into two distinct DataFrames based on the `Vehicle_Type`. A scatter plot maps `Expected_Ride_Duration` against `Historical_Cost_of_Ride`. Both vehicle types are plotted on the same axes using distinct colors to compare how pricing scales with time.
- Cost by Booking Time: The average `Historical_Cost_of_Ride` is calculated by grouping the data by the ordered booking times. The output is a bar chart formatted to display currency values.

- **Cost per Minute by Region:** A multi-level grouping operation aggregates the data by both `Location_Category` and `Vehicle_Type` to compute the mean `Cost_per_Minute`. The resulting table is unstacked and visualized as a multi-line plot, demonstrating how per-minute pricing efficiency fluctuates across regions for both vehicle tiers.

Prediction Module

Categorical variables including `Vehicle_Type`, `Location_Category`, `Time_of_Booking`, and `Customer_Loyalty_Status` are transformed into numerical formats using one-hot encoding via the `pd.get_dummies` function. The first category is dropped during this process to prevent multicollinearity.

Following this, the target variable (`y`) is explicitly defined as the `Historical_Cost_of_Ride`. The feature matrix (`X`) is constructed by isolating the predictive features, which involves dropping the target variable and the engineered `Cost_per_Minute` column. Then the processed dataset is partitioned into training and testing sets using a 70/30 split (`test_size=0.3`), ensuring the model is evaluated on unseen data. A fixed random state (`random_state=42`) is applied to guarantee reproducible results.

Algorithm Engine and Model Training

The core prediction engine is built using a `RandomForestRegressor` instantiated from `sklearn.ensemble` module. After this, the model is configured with `n_estimators=100` (establishing 100 individual decision trees) and a `max_depth` of 8 to mitigate the risk of overfitting. Lastly, the algorithm is trained by fitting the Random Forest model exclusively on the training subset using `random_forest_model.fit(X_train, y_train)`.

Findings and Managerial Insights

This analysis is based on a comprehensive dataset comprising 1000 individual ride records from a ride-sharing service. The scope of our investigation covers various operational dimensions, including driver availability, rider demand, geographic locations, and historical pricing structures. By applying advanced data cleaning and a Random Forest regression model, we isolated the primary drivers of cost and identified critical periods of supply-demand imbalance.

Data Overview & Operational Metrics

The exploratory data analysis and predictive modeling revealed several crucial patterns regarding market behavior and pricing dynamics:

- **Price and Ride Duration Relationship:** Pricing for Economy and Premium vehicles scales linearly for short durations using similar core formulas. However, longer trips exhibit significant price variance, indicating that extended rides are highly susceptible to external variables.

- **Demand and Supply Dynamics:** There is a persistent high demand for economic vehicles that peaks significantly at night. Paradoxically, the Economy tier has lower vehicle availability compared to the Premium fleet, creating a structural shortage.
- **Critical Operational Imbalance:** The platform faces extreme pressure during night hours, where the Demand/Supply ratio for Economy vehicles reaches a severe peak of 3.69.
- **Model Reliability and Accuracy:** The predictive model accounts for 84.68% of price variation (R^2 score). With a Mean Absolute Error (MAE) of approximately \$55 (roughly 15% of the average ride cost), the model is considered highly reliable for the majority of market conditions.
- **Primary Pricing Factors (Feature Importance):** Expected ride duration is the dominant predictor, representing 90.6% of the model's feature importance. User ratings and the supply-demand ratio serve as secondary but meaningful factors.

Strategic Recommendations

Transitioning from static to algorithmic pricing allows for more agile market responses. Based on the 85% confidence level of the current model, the following strategic actions are recommended to optimize platform efficiency:

- **Incentivize Economy Drivers:** To mitigate the severe 3.69 Demand/Supply imbalance at night, management should implement a dynamic base pay increase for drivers specifically during night shifts to boost active supply.
- **Dynamic Premium Discounting:** To prevent customer loss during Economy vehicle shortages, the platform should introduce automated discounting for Premium rides when Economy demand exceeds capacity, shifting load to the underutilized Premium fleet.
- **Strategic Capacity Alignment:** The findings suggest a fundamental need to increase the baseline volume of Economy cars during peak night hours through recruitment or fleet reallocation.
- **Future Model Iterations:** To bridge the remaining 15% accuracy gap, future iterations should integrate real-time external data feeds, such as local weather patterns and traffic volume, to refine the predictive cost engine.

Managerial Considerations

Before proceeding with the proposed strategies, the underlying trade-offs and technical constraints inherent in the current analysis must be considered:

- **Strategic Trade-offs:** Implementing dynamic base pay increases for night-shift drivers involves a critical balance between maintaining driver retention and overall platform profitability. While higher pay solves supply shortages, it must be carefully indexed to ensure it does not erode margins during lower-demand intervals.
- **Implementation Limitations:** It is essential to remain mindful of the 15% accuracy variance within the model. Furthermore, the reliance on historical data means that the

model may not immediately account for sudden shifts in market behavior or unexpected events until those patterns are integrated into new training cycles.

Team-Member Roles

- Lucas Krämer: Findings, Visualizations, and Algorithm training
- Richard Horvath: Data cleaning, bug fixing
- Aijia Liu: Model evaluation
- Adam Pasálek: Dashboard (frontend), Business Implications, section Findings and Managerial Insights in the report
- Zhixian Huang: Utilities optimization